JPA/Hibernate Performance Tuning

Chapter 04 - Sub-Chapter 10

Beginner-Friendly Guide with Examples, Diagrams and Q&A;

SQL Logging & Statistics	show-sql, generate_statistics
Batch Inserts/Updates	jdbc.batch_size, SEQUENCE strategy
Pagination	Pageable, LIMIT/OFFSET
DTO Projections	Interface projections, constructor JPQL
Index Strategy	@Index annotation, EXPLAIN plan
HikariCP Pool Tuning	maximumPoolSize, connectionTimeout
Common Anti-Patterns	SELECT *, N+1, missing indexes

Performance Checklist Overview

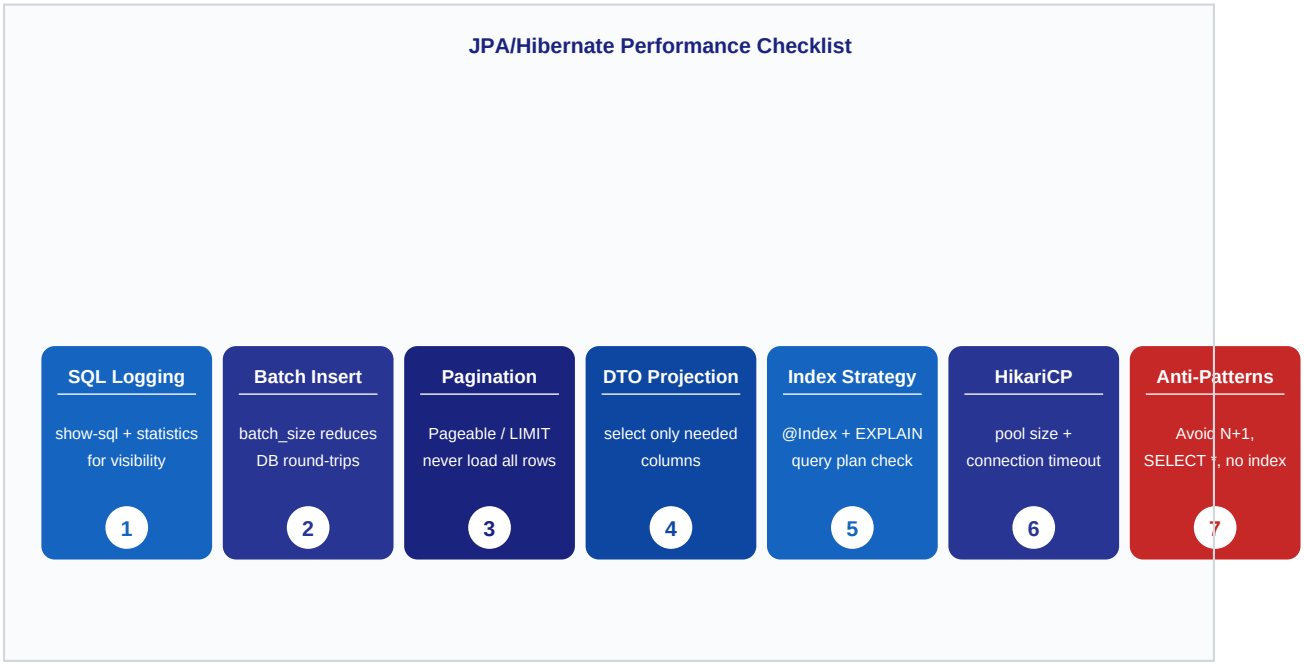

Figure 1: Seven performance techniques — each numbered in order of impact.

Batch Insert: Loop vs Batch

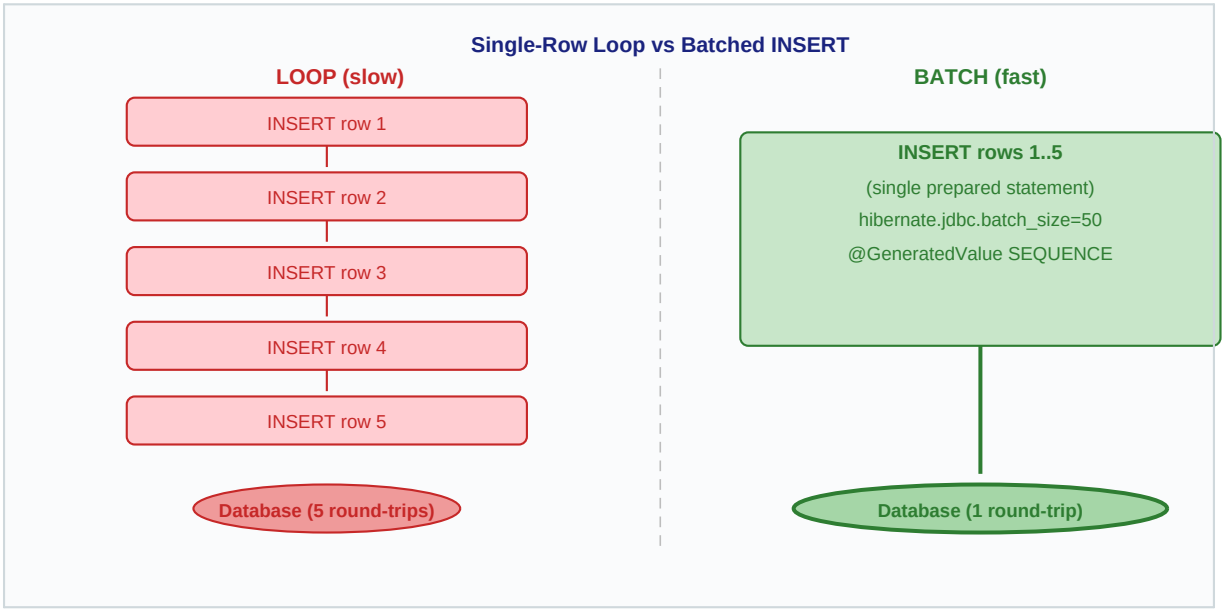

Figure 2: Loop inserts cause N round-trips; batched inserts use 1 round-trip.

1 SQL Logging and Hibernate Statistics

spring.jpa.show-sql | hibernate.generate_statistics

Before tuning anything, you need visibility. Enabling SQL logging prints every query Hibernate sends to the database. Hibernate statistics go further: they count queries, cache hits, entity loads, and more. Use these in development to spot problems before they reach production.

application.properties

```
# Show SQL in console
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Enable Hibernate statistics
spring.jpa.properties.hibernate.generate_statistics=true
logging.level.org.hibernate.stat=DEBUG
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql=TRACE
```

Reading the statistics output

```
// Inject EntityManagerFactory and unwrap SessionFactory
SessionFactory sf = emf.unwrap(SessionFactory.class);
Statistics stats = sf.getStatistics();
stats.setStatisticsEnabled(true);

long queryCount = stats.getQueryExecutionCount();
long entityFetch = stats.getEntityFetchCount();
System.out.println("Queries executed: " + queryCount);
System.out.println("Entities fetched: " + entityFetch);
```

Key Rules

- Enable show-sql only in dev/test — it adds console I/O overhead.
- Use hibernate.format_sql=true to make multi-line queries readable.
- generate_statistics is safe in staging; disable in high-throughput prod.
- Watch for getQueryExecutionCount spikes — sign of N+1 problems.
- Pair with a query logger like P6Spy for full parameter values.

Q1. What does spring.jpa.show-sql do?

It tells Hibernate to print each SQL statement to the console (stdout). It does NOT show bind parameter values — add logging.level for that.

Q2. What is hibernate.generate_statistics?

A flag that activates internal counters for queries, cache hits, flushes, and entity loads. Read them via SessionFactory.getStatistics().

Q3. Why should show-sql be off in production?

Every query adds a System.out write on the application thread. Under load this becomes a bottleneck. Use a log appender at DEBUG level instead.

Q4. How do I see parameter values in logged SQL?

Add: logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE This prints each bound value next to its placeholder.

Q5. What counter signals an N+1 query problem?

EntityFetchCount rising steeply relative to QueryExecutionCount is the classic signal. One query loads a list; then N extra queries load associations.

2 Batch Inserts and Updates

hibernate.jdbc.batch_size | @GeneratedValue SEQUENCE

Without batching, Hibernate sends one SQL INSERT per entity. Saving 1000 records means 1000 round-trips to the database. JDBC batch support groups those statements into a single network call. The critical rule: use SEQUENCE (not IDENTITY) for primary keys, because Hibernate must know the ID before batching.

application.properties

```
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
spring.jpa.properties.hibernate.jdbc.batch_versioned_data=true
```

Entity with SEQUENCE strategy

```
@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
                    generator = "product_seq")
    @SequenceGenerator(name = "product_seq",
                       sequenceName = "product_seq",
                       allocationSize = 50)
    private Long id;

    private String name;
    private double price;
}
```

Key Rules

- batch_size = 50 is a common starting point; tune based on row width.
- IDENTITY strategy disables batching — always prefer SEQUENCE.
- order_inserts/order_updates groups statements of the same type together.
- Flush and clear the EntityManager every batch_size iterations in bulk loops.
- allocationSize on the sequence generator should match batch_size to reduce sequence calls.

Q1. Why does IDENTITY disable batch inserts?

With IDENTITY, the database generates the ID after the INSERT. Hibernate needs the ID immediately to build its first-level cache, so it cannot delay or group the statement.

Q2. What does order_inserts=true do?

It reorders pending INSERT statements so that all inserts for the same table are grouped. This allows JDBC to batch them in one executeBatch() call.

Q3. How do I flush in a bulk save loop?

Every batch_size iterations call entityManager.flush() then entityManager.clear(). flush() sends SQL; clear() frees memory.

Q4. What is allocationSize on @SequenceGenerator?

Hibernate fetches sequence values in blocks of allocationSize. Setting it to 50 means one sequence call per 50 entities, reducing database round-trips.

Q5. Can I use batch updates too?

Yes. Set order_updates=true and jdbc.batch_versioned_data=true. Hibernate then batches UPDATE statements the same way as INSERTs.

3 Pagination

Never load all rows -- always use Pageable / LIMIT / OFFSET

Loading every row from a large table into memory is one of the most common performance mistakes. Pagination limits the result set at the database level using LIMIT and OFFSET (or equivalent). Spring Data JPA makes this trivial with the Pageable interface — no manual SQL needed.

Repository with Pageable

```
public interface ProductRepository
        extends JpaRepository<Product, Long> {

    // Spring Data adds LIMIT/OFFSET automatically
    Page<Product> findByCategory(String category, Pageable pageable);
}
```

Service usage

```
@Service
public class ProductService {
    @Autowired ProductRepository repo;

    public Page<Product> getPage(int page, int size) {
        Pageable p = PageRequest.of(page, size,
                        Sort.by("name").ascending());
        return repo.findByCategory("Electronics", p);
    }
}
```

Key Rules

- Never call findAll() on a table that can grow large.
- PageRequest.of(page, size) is zero-indexed — page 0 is the first page.
- Always add a Sort to get deterministic results across pages.
- OFFSET pagination degrades on large offsets — consider keyset pagination for deep pages.
- Expose page and size as request params in REST controllers with sensible defaults.

Q1. What SQL does Pageable generate?

Spring Data appends LIMIT {size} OFFSET {page * size} to the query, so only the requested slice is fetched from the database.

Q2. What is the difference between Page and Slice?

Page runs an extra COUNT(*) query to get the total count. Slice skips the count query — use it when you only need 'has next page' logic.

Q3. Why does OFFSET become slow on large tables?

The database still reads and discards all rows before the offset. For page 1000 at size 20 it discards 20 000 rows. Keyset (cursor) pagination avoids this.

Q4. How do I paginate a native query?

Annotate with @Query(nativeQuery=true) and add a countQuery parameter. Spring Data uses the count query for the Page total.

Q5. What default page size should I use?

20 is a common REST default. Enforce a maximum (e.g. 100) server-side so clients cannot request unbounded result sets.

4 DTO Projections

Select only needed columns -- interface or constructor projections

Loading a full entity fetches every column even when you only need two fields for a list screen. DTO projections tell Hibernate to SELECT only the columns you declare, reducing data transfer, memory, and mapping overhead. Spring Data supports both interface-based and constructor-expression projections.

Interface-based projection

```
// Declare only the fields you need
public interface ProductSummary {
    Long getId();
    String getName();
    double getPrice();
}

// Repository -- Hibernate selects id, name, price only
public interface ProductRepository
        extends JpaRepository<Product, Long> {
    List<ProductSummary> findAllProjectedBy();
}
```

Constructor expression (JPQL)

```
@Query("SELECT new com.example.dto.ProductDto(p.id, p.name, p.price) " +
       "FROM Product p WHERE p.category = :cat")
List<ProductDto> findSummaries(@Param("cat") String category);
```

Key Rules

- Interface projections work with Spring Data method names -- no @Query needed.
- Constructor projections are explicit and type-safe -- preferred for complex queries.
- Never load entity graphs when only IDs or counts are needed.
- Projections skip Hibernate's dirty-checking overhead -- they are read-only by design.
- Use @Value in interface projections for computed fields: @Value("#{target.price * 1.2}").

Q1. What SQL does an interface projection generate?

Hibernate inspects the interface getter names and emits SELECT id, name, price instead of SELECT *. Only the declared columns are fetched.

Q2. Can I mix projections and pagination?

Yes. Return Page and pass Pageable as normal. Both features compose cleanly in Spring Data.

Q3. When should I use a constructor projection over an interface?

When you need computed columns, aggregations, or JOIN results that map to a specific DTO class. Interface projections work best for simple entity fields.

Q4. Do projections bypass the first-level cache?

Yes. Projected results are not managed entities, so they are not stored in the persistence context. This reduces memory but means no dirty tracking.

Q5. Can I return a projection from a native query?

Yes. Define the interface with getters matching the column aliases in your native SQL, and Spring Data maps them automatically.

5 Index Strategy

@Index annotation | Query plan analysis with EXPLAIN

A query without a matching index causes a full table scan — the database reads every row to find matches. Adding the right index reduces a scan from O(n) to O(log n). JPA lets you declare indexes in @Table, and you can inspect the query plan with EXPLAIN (MySQL/PostgreSQL) to verify they are used.

Declaring indexes with @Table

```
@Entity
@Table(name = "product",
       indexes = {
           @Index(name = "idx_product_category",
                  columnList = "category"),
           @Index(name = "idx_product_price_category",
                  columnList = "price, category"),
           @Index(name = "uq_product_sku",
                  columnList = "sku", unique = true)
       })
public class Product {
    @Id @GeneratedValue private Long id;
    private String category;
    private double price;
    private String sku;
}
```

Checking the query plan (PostgreSQL)

```
-- Run in psql or any SQL client
EXPLAIN ANALYZE
SELECT * FROM product WHERE category = 'Electronics';

-- Look for: Index Scan vs Seq Scan
-- Index Scan = index is being used (fast)
-- Seq Scan   = full table scan (slow for large tables)
```

Key Rules

- Index columns you filter on (WHERE) or sort on (ORDER BY) frequently.
- Composite indexes: put the most selective column first.
- Unique indexes enforce uniqueness AND speed up lookups.
- Too many indexes slow down INSERT/UPDATE -- index only what queries need.
- Re-run EXPLAIN after adding an index to confirm it is chosen by the planner.

Q1. Does @Index create the index automatically?

Only when Hibernate's DDL auto is set to create or update (spring.jpa.hibernate.ddl-auto=update). In production, manage indexes via migration tools like Flyway or Liquibase.

Q2. What is a composite index?

An index on multiple columns, e.g. (price, category). It accelerates queries that filter on the leading column(s) of the index.

Q3. When does an index hurt performance?

Every INSERT, UPDATE, or DELETE on an indexed column must also update the index structure. Tables with very high write rates may see overhead from too many indexes.

Q4. What does Seq Scan in EXPLAIN mean?

Sequential scan: the database reads every row in the table. Normal for very small tables, but a red flag on large tables where an index should apply.

Q5. How do I add an index without downtime in PostgreSQL?

Use CREATE INDEX CONCURRENTLY — it builds the index without locking writes. This is preferred for production tables with live traffic.

6 HikariCP Connection Pool Tuning

maximumPoolSize | connectionTimeout | idleTimeout

Every database operation needs a connection. Creating a new TCP connection is expensive (milliseconds). A connection pool keeps a set of open connections ready to reuse. HikariCP is Spring Boot's default pool and is the fastest available. Tuning the pool size and timeouts prevents both starvation and resource waste.

application.properties

```
# Maximum connections kept in the pool
spring.datasource.hikari.maximum-pool-size=20

# Minimum idle connections maintained
spring.datasource.hikari.minimum-idle=5

# Milliseconds to wait for a free connection before throwing
spring.datasource.hikari.connection-timeout=30000

# Milliseconds an idle connection lives before being closed
spring.datasource.hikari.idle-timeout=600000

# Max lifetime of any connection (rotate before DB closes it)
spring.datasource.hikari.max-lifetime=1800000
```

Verifying pool behaviour (Actuator)

```
# application.properties -- expose metrics endpoint
management.endpoints.web.exposure.include=metrics,health

# Then call:
# GET /actuator/metrics/hikaricp.connections.active
# GET /actuator/metrics/hikaricp.connections.pending
```

Key Rules

- maximumPoolSize = (CPU cores * 2) + disk spindle count is a common formula.
- Never set maximum-pool-size to hundreds -- database handles far fewer concurrent connections.
- connection-timeout should be below your HTTP request timeout.
- max-lifetime must be less than the database's wait_timeout to prevent stale connections.
- Monitor hikaricp.connections.pending -- sustained > 0 means pool is undersized.

Q1. What happens when all connections are in use?

HikariCP waits up to connectionTimeout milliseconds for a free connection. If none becomes available, it throws SQLTimeoutException.

Q2. What is the right pool size?

HikariCP's own guidance: (core_count * 2) + effective_spindle_count. For a 4-core machine with SSD this is often 10. Benchmark to confirm.

Q3. Why does max-lifetime matter?

Databases close idle connections after wait_timeout. If Hibernate holds a connection longer, the next query on it fails. max-lifetime keeps the pool rotating connections before that happens.

Q4. How does minimum-idle affect performance?

Keeping minimum-idle warm connections avoids creation latency on burst traffic. For constant-load services, set minimum-idle equal to maximumPoolSize.

Q5. Where do I see pool metrics at runtime?

Enable spring-boot-actuator and check /actuator/metrics/hikaricp.connections.*. The pending and timeout counters are the most important ones to watch.

7 Common Anti-Patterns

SELECT * | N+1 | Unnecessary associations | Missing indexes

Most JPA performance problems fall into four patterns. Recognising them early saves hours of debugging. The N+1 problem is the most common: one query loads a list, then Hibernate fires one extra query per item to load a related entity.

Anti-pattern 1: SELECT * (full entity load)

```
// BAD -- loads every column even if you only need name
List<Product> all = productRepository.findAll();

// GOOD -- projection loads only what you need
List<ProductSummary> names = productRepository.findAllProjectedBy();
```

Anti-pattern 2: N+1 Query

```
// BAD -- 1 query for orders + N queries for each customer
List<Order> orders = orderRepository.findAll();
for (Order o : orders) {
    System.out.println(o.getCustomer().getName()); // lazy load!
}

// GOOD -- single JOIN FETCH loads everything at once
@Query("SELECT o FROM Order o JOIN FETCH o.customer")
List<Order> findAllWithCustomer();
```

Anti-pattern 3: Loading unnecessary associations

```
// BAD -- EAGER fetch always loads items even when not needed
@OneToMany(fetch = FetchType.EAGER)
private List<OrderItem> items;

// GOOD -- LAZY is the default; load only when accessed
@OneToMany(fetch = FetchType.LAZY)
private List<OrderItem> items;
```

Anti-pattern 4: Missing indexes on foreign keys

```
// Add index on the FK column that you JOIN on
@Entity
@Table(name = "order",
       indexes = @Index(name = "idx_order_customer_id",
                        columnList = "customer_id"))
public class Order {
    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

Key Rules

- Default fetch type for @OneToMany and @ManyToMany is LAZY -- keep it that way.
- Use JOIN FETCH or @EntityGraph to load associations only when you actually need them.
- Run hibernate.generate_statistics to detect N+1 (watch EntityFetchCount).
- Always index foreign key columns used in JOIN conditions.
- Avoid SELECT * in native queries -- list columns explicitly.

Q1. How do I detect an N+1 problem without reading code?

Enable hibernate.generate_statistics. If EntityFetchCount is close to the number of rows in your list result, you have N+1.

Q2. What is @EntityGraph and how does it help?

It lets you define named fetch plans on an entity. At the repository method level you annotate with @EntityGraph(attributePaths = {"customer"}) to trigger a JOIN FETCH without changing the entity mapping.

Q3. Is EAGER fetch type ever appropriate?

Only when you always need the association and it is small (e.g. a single ManyToOne to a reference table). For collections, EAGER is almost always wrong.

Q4. Why index foreign key columns specifically?

JOIN conditions and WHERE filters on FK columns are extremely common. Without an index the database must scan the entire child table for each parent row.

Q5. What tool helps find missing indexes automatically?

Most databases have a slow query log and EXPLAIN plans. Tools like pg_stat_user_indexes (PostgreSQL) show unused indexes, and pg_stat_user_tables shows sequential scan counts as index-missing signals.